WildfireWatch Rwanda

Explanation of the ERD and Trigger Diagram

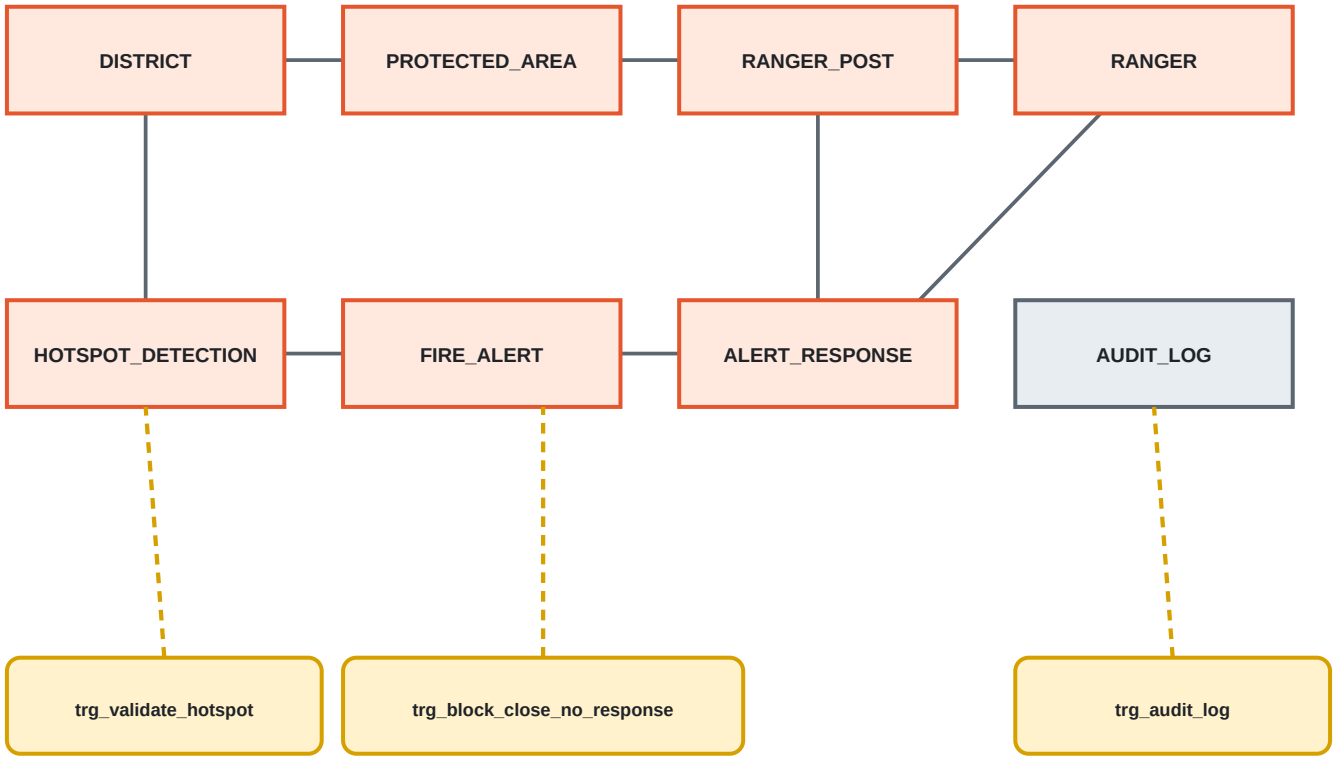
Shema Vainqueur | Student ID 20251SEN313 | INSY 8311 (PL/SQL) | Instructor: Eric Maniraguha

Companion to WildfireWatch_Rwanda_ERD_Triggers.drawio. Status: Phase 0 draft; columns will be finalised when the data is normalised to 3NF in Phase II.

1. What the diagram shows

The diagram is the planned structure of the WildfireWatch Rwanda database. It answers three questions: which tables will hold the data, how the tables are linked, and which triggers protect the data. The project turns NASA FIRMS satellite fire detections into alerts that are assigned to a ranger post and tracked until closed.

How to read it. Each box is a table. PK marks a primary key (the column that identifies a row). FK marks a foreign key (a column that points to a row in another table). Lines between tables are relationships; a crow's foot at one end means "many". Yellow boxes are triggers, and a dashed line joins each trigger to the table it guards.



Orange boxes = tables. Yellow boxes = triggers (dashed line = table the trigger guards). Solid lines = foreign-key relationships.

Simplified schematic of the draw.io diagram (column details are in the .drawio file and in section 2).

2. The eight tables

Table	What it stores	Key columns (draft)	Linked to
DISTRICT	Rwanda districts, used to report fires by district	district_id (PK), name, province, centre_lat, centre_lon	Protected areas and hotspots
PROTECTED_AREA	Parks and forests (simulated boundaries)	area_id (PK), name, area_type, district_id (FK)	District; ranger posts
RANGER_POST	Ranger stations used for nearest-post assignment (simulated)	post_id (PK), name, latitude, longitude, area_id (FK), district_id (FK)	Protected area; rangers; responses
RANGER	Staff based at a post	ranger_id (PK), full_name, post_id (FK)	Ranger post; responses
HOTSPOT_DETECTION	Each real FIRMS detection loaded from the CSV	hotspot_id (PK), latitude, longitude, acq_date, acq_time, brightness, confidence, frp, satellite, daynight, district_id (FK), area_id (FK)	District; fire alert
FIRE_ALERT	An alert opened for a detection, with its risk level and status	alert_id (PK), hotspot_id (FK), risk_level, status, opened_at, closed_at	Hotspot; responses
ALERT_RESPONSE	What was done about an alert: which post, who, when, outcome	response_id (PK), alert_id (FK), post_id (FK), ranger_id (FK), arrival_time, outcome	Alert; post; ranger
AUDIT_LOG	Record of every insert, update and delete on the main tables	log_id (PK), table_name, action, record_id, old_value, new_value, changed_by, changed_at	Written by trigger

3. Relationships

Relationship	Meaning
DISTRICT 1 to many PROTECTED_AREA	A district can contain several parks or forests.
PROTECTED_AREA 1 to many RANGER_POST	A park has one or more ranger posts.
RANGER_POST 1 to many RANGER	Each post has several rangers.
DISTRICT 1 to many HOTSPOT_DETECTION	Each detection falls in one district; a district has many detections.
HOTSPOT_DETECTION to FIRE_ALERT	A detection may open an alert. The business rule is at most one alert per detection; the diagram draws the line as many-sided, so the one-alert limit will be enforced in PL/SQL.
FIRE_ALERT 1 to many ALERT_RESPONSE	An alert can have more than one response, for example a second visit.
RANGER_POST and RANGER 1 to many ALERT_RESPONSE	Each response records which post and which ranger went.

Known gaps to fix in Phase II: HOTSPOT_DETECTION.area_id and RANGER_POST.district_id are listed as foreign keys, but the diagram does not yet draw their lines. Also, because the data note says nearby same-day detections are probably one fire, one alert may need to cover several detections; the single hotspot_id in FIRE_ALERT may become a link table.

4. The triggers

Trigger	Fires on	What it does	Why it is needed
trg_validate_hotspot	BEFORE INSERT or UPDATE on HOTSPOT_DETECTION	Rejects coordinates outside Rwanda and confidence outside 0-100, and raises the invalid location exception.	Keeps bad rows out of the database at the point of entry, whichever tool loads the data.
trg_block_close_no_response	BEFORE UPDATE on FIRE_ALERT	Blocks changing the status to CLOSED when no matching row exists in ALERT_RESPONSE.	An alert cannot be closed without evidence that someone responded, so response time stays meaningful.
trg_audit_log	AFTER INSERT, UPDATE or DELETE on HOTSPOT_DETECTION, FIRE_ALERT and ALERT_RESPONSE	Writes the table, action, record, old and new values, user and time to AUDIT_LOG.	Gives a full history of every change, which meets the audit objective.

Example. The 2025 FIRMS file has a row at -1.5131, 30.4176 (2025-07-17) with confidence 0 and fire power 133.2. Confidence 0 is inside the allowed range, so *trg_validate_hotspot* would accept it. Such rows are handled by the risk scoring function calc_risk_level and by checks in load_hotspots, not by this trigger.

Exceptions that are not triggers. Duplicate detection is raised by load_hotspots, and unassigned post is raised by open_alert when no ranger post can be found.

5. How the pieces work together

Step	What happens	Objects involved
1	The daily DBMS_SCHEDULER job runs and calls load_hotspots to import new FIRMS rows.	Scheduler, load_hotspots
2	Each new row is checked; invalid rows are rejected.	trg_validate_hotspot, HOTSPOT_DETECTION
3	open_alert scores the detection, opens an alert and assigns the nearest ranger post.	calc_risk_level, open_alert, FIRE_ALERT
4	A ranger responds; arrival time and outcome are recorded.	ALERT_RESPONSE, response_minutes
5	close_alert closes the alert only if a response exists.	close_alert, trg_block_close_no_response
6	Every change in steps 2-5 is logged; unanswered alerts are flagged by the job.	trg_audit_log, AUDIT_LOG

Limitations. Ranger posts and park boundaries are simulated reference tables, not real operational data. MODIS detections are 1 km pixels and are not confirmed fires. All column lists are drafts.